AetherOps

AI-Powered Cloud Security, Cost Intelligence
and Multi-Cloud Comparison Platform

Project Type	B.Tech 3rd Year Project
Institution	CHARUSAT University — DEPSTAR
Team	Tushya Patel · Priyanshi Pujara · Janki Panchal · Dhruv Mistry
Domain	Cloud Security · AI Integration · DevSecOps
Version	1.0 | September 2026

This document describes the complete design, architecture, feature set, and implementation plan for AetherOps — a cloud security and AI intelligence platform built as a student capstone project. Every feature described here is scoped to be genuinely buildable by a four-person student team within an academic year.

Table of Contents

1.	**Project Overview**	What AetherOps is and why it exists
2.	**The Problem We Are Solving**	Real pain points in cloud management
3.	**What We Are Building**	Full feature list and scope
4.	**System Architecture**	How all the pieces connect
5.	**Cloud Domain**	Resource discovery, billing, multi-cloud comparison
6.	**Security Domain**	Rules engine, attack paths, RBAC, audit
7.	**AI Domain**	Report generation, chat bot, smart assistant
8.	**Technology Stack**	Every tool and why we chose it
9.	**Database Schema**	Tables, fields, and relationships
10.	**API Reference**	Key endpoints and their purpose
11.	**Implementation Plan**	Week-by-week roadmap
12.	**Feasibility Assessment**	Honest effort and risk ratings
13.	**What This Demonstrates**	Resume and domain value

1. Project Overview

AetherOps is a cloud security, billing intelligence, and AI assistant platform built for AWS. A user connects their AWS account through a read-only IAM role, and AetherOps automatically discovers every resource in that account — servers, storage, databases, identity configurations — along with how those resources relate to each other. It then evaluates every resource against a defined set of security rules, links related findings into realistic attack paths, and uses an AI model to generate a full plain-language report of the account's security posture, cost breakdown, and multi-cloud comparison.

The project is aimed at students, freelancers, small teams, and early-stage startups who manage real AWS infrastructure but cannot afford enterprise-grade tools like Wiz or Orca. AetherOps gives them the same core capabilities — posture management, attack-path correlation, AI explanation, and cost intelligence — in a free, self-hostable package.

One-line summary:

> Connect your AWS account → get a complete security and cost report, with AI-narrated attack paths and a comparison of what the same workload would cost on Azure or GCP — all explained in plain English.

2. The Problem We Are Solving

Anyone who has set up an AWS account eventually runs into the same set of frustrations:

- **Too many warnings, no priority.** A security scan returns forty findings with no clear indication of which three are actually dangerous and which thirty-seven can wait.
- **No connected view.** Each finding is reported in isolation. A stale IAM key, an open security group, and a public S3 bucket are each flagged separately — but no tool tells you that all three form one attack path.
- **Unexpected costs.** Forgotten test servers, oversized instances, and idle resources silently accumulate charges with no clear visibility into what is driving spend.
- **No cross-cloud perspective.** A team starts on AWS because it is familiar, but has no easy way to know whether their workload would be meaningfully cheaper or better-suited on Azure or GCP.
- **Jargon-heavy output.** Even when findings are surfaced, they are described in technical language that is difficult to act on without deep security knowledge.

> The core problem is not a lack of data — AWS surfaces extensive detail about everything in an account. The real gap is turning that raw detail into a clear, prioritised, actionable understanding of what is wrong and why. AetherOps exists to close that gap.

3. What We Are Building

AetherOps is built around four confirmed feature areas. Every feature listed here is within the scope of a four-person student team working over one academic year.

- **AWS Resource Discovery**
  Automatic inventory of EC2, S3, RDS, IAM, Lambda — and the relationships between them — pulled via a read-only IAM role using boto3.

- **Security Evaluation Engine**
  YAML-defined rules check each resource for common misconfigurations. Each violation becomes a finding with a severity rating (High / Medium / Low).

- **Attack-Path Correlation**
  Related findings are linked into realistic attack chains using graph traversal (BFS/DFS on the relationships table). Shown as a dedicated view, separate from raw findings.

- **Billing Intelligence**
  AWS Cost Explorer integration shows monthly cost per service with idle resource flagging.

- **AI-Generated Full Report**
  When the user clicks Smart Assistant, the system aggregates all findings, paths, and billing data and sends them to Gemini, which returns a structured, plain-language report displayed on the dashboard.

- **Multi-Cloud Cost Comparison**
  Live pricing is fetched from AWS Pricing API, Azure Retail Prices API, and GCP Billing Catalog API. A SKU equivalency table maps instance types across providers. The AI interprets the delta and gives qualitative migration guidance.

- **Contextual Chat Bot**
  A small chat widget scoped to the user's own report. Every message includes the full report JSON in the system prompt, so answers are grounded in real account data — not generic cloud advice.

- **Role-Based Access Control**
  Three roles: Admin (full access, triggers scans), Auditor (read + audit log), Viewer (dashboard read-only). JWT-based authentication.

- **Audit Trail**
  Append-only log of every login, scan, finding resolution, and role change.

4. System Architecture

The system is organised into five layers. Each layer depends only on the one directly beneath it, which keeps the design modular — changing how resources are stored, for example, does not require any change to the dashboard.

Presentation	*React + Tailwind CSS*	Dashboard: resource inventory, findings, attack paths, AI report, chat bot, cost comparison.
Application	*FastAPI (Python)*	Authentication, RBAC enforcement, rule evaluation engine, AI orchestration, pricing API calls.
Data	*PostgreSQL + SQLAlchemy*	Resources, relationships, findings, attack paths, costs, users, audit logs.
Cloud Integration	*boto3 + AWS APIs*	Read-only connection to the customer's AWS account. All discovery and billing data comes through here.
AI	*Google Gemini API*	Report generation, attack-path narration, cloud comparison interpretation, chat bot responses.

End-to-End Data Flow

The sequence below shows how a single user interaction — clicking Generate Report — moves through every layer of the system:

1	**User connects AWS account**	Provides an IAM Role ARN. AetherOps assumes the role via STS.
2	**Resource Discovery**	boto3 calls describe_instances(), list_buckets(), list_users(), describe_db_instances(), list_functions(). All resources and their relationships stored in PostgreSQL.
3	**Security Evaluation**	Rules Engine loads YAML rules, evaluates each resource, writes findings (id, resource, severity, rule, status) to the findings table.
4	**Attack-Path Correlation**	NetworkX graph is built from the relationships table. BFS from high-severity nodes identifies chains. Paths stored as JSON in attack_paths table.
5	**Billing Pull**	Cost Explorer API called for current month. Per-service cost stored in costs table.
6	**Pricing API Fetch**	Parallel calls to AWS Pricing API, Azure Retail Prices API, GCP Billing Catalog API. SKU equivalency table maps instance types across clouds.
7	**AI Report Generation**	Aggregated data sent to Gemini in a single structured prompt. Gemini returns a JSON report: security summary, attack narratives, cost breakdown, cloud comparison, recommended actions.
8	**Dashboard Render**	React renders the report sections. Report saved to DB linked to user and scan timestamp.

9	**Chat Bot Activated**	Report JSON injected into Gemini system prompt. User asks follow-up questions. Every answer is grounded in their actual account data.

5. Cloud Domain

5.1 Resource Discovery

Discovery is the foundation of everything. Without an accurate, relationship-aware inventory, neither the security evaluation nor the attack-path correlation can work correctly.

What we discover:

Resource Type	AWS Service	Key Attributes Captured
EC2 Instances	ec2.describe_instances()	Instance ID, type, state, security groups, VPC, AMI, key pair
S3 Buckets	s3.list_buckets() + get_bucket_acl()	Name, ACL, policy, public access block settings
RDS Databases	rds.describe_db_instances()	Engine, publicly accessible flag, encryption status
IAM Users & Keys	iam.list_users() + list_access_keys()	User, MFA status, key age, last used date
Security Groups	ec2.describe_security_groups()	Inbound/outbound rules, port ranges, CIDR blocks
Lambda Functions	lambda.list_functions()	Runtime, role, VPC config, timeout
VPCs & Subnets	ec2.describe_vpcs()	CIDR, internet gateway attachment, route tables

Critically, we also record the relationships between resources — which security group is attached to which EC2, which IAM role can reach which resource — using a separate relationships table. This is what makes attack-path correlation possible later.

5.2 Billing Intelligence

Using the AWS Cost Explorer API (ce.get_cost_and_usage()), we pull the current month's spend broken down by service. This populates the costs table and is displayed on the dashboard as a cost breakdown chart. The AI report includes a plain-language commentary on cost drivers.

Data Point	Source	Dashboard Use
Monthly cost per service	Cost Explorer GetCostAndUsage	Bar chart, cost table
Day-by-day spend trend	Cost Explorer with DAILY granularity	Line chart
Idle resource flags	Cross-reference utilisation vs cost	Warning badge on resource
Top 3 cost drivers	Sorted cost table	Summary card in AI report

5.3 Multi-Cloud Cost Comparison

This is one of AetherOps's differentiating features. When the AI report is generated, the system also fetches live pricing from all three major cloud providers and compares the user's current AWS spend against equivalent configurations on Azure and GCP.

How the pricing pipeline works:

Source	Endpoint / Method	Notes

AWS Pricing API	*https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/index.json*	Free, no key required. Returns on-demand pricing per instance type per region.
Azure Retail Prices API	*https://prices.azure.com/api/retail/prices*	Free, no key required. Queryable by SKU name and region.
GCP Billing Catalog API	*https://cloudbilling.googleapis.com/v1/services*	Requires a GCP API key. Free quota is sufficient for this use case.
SKU Equivalency Table	*Static mapping file (JSON)*	Maps AWS instance types to nearest Azure and GCP equivalents. Community-maintained mapping tables are available on GitHub.
AI Interpretation	*Gemini prompt with pricing delta*	AI explains the cost difference and whether migration is advisable given the user's actual utilisation pattern.

Important scope note: we are not building a cost calculator. We are using real pricing data to give the AI accurate numbers, and letting the AI give qualitative migration guidance. The output is: 'Your t3.medium workload at 15% average CPU utilisation would cost approximately X on Azure B2s vs Y on AWS — migration is likely cost-neutral but GCP offers better egress pricing for Asia-Pacific traffic.' This is honest, useful, and buildable.

6. Security Domain

6.1 Rules Engine

The rules engine is the core of the security evaluation. It loads rules from YAML files, evaluates each resource against applicable rules, and writes findings to the database. YAML was chosen because rules are readable without code knowledge and can be updated without touching application logic.

YAML rule structure:

```
rule_id: SG-001 title: Security group open to internet on SSH port resource: security_group severity: HIGH condition: field: inbound_rules contains: port: 22 cidr: 0.0.0.0/0 remediation: Restrict inbound SSH to known IP ranges only.
```

Current rule set:

Rule ID	What It Checks	Severity
SG-001	Security group open on port 22 or 3389 to 0.0.0.0/0	HIGH
S3-001	S3 bucket publicly readable or writable	HIGH
S3-002	S3 bucket with no server-side encryption	MEDIUM
IAM-001	IAM user with no MFA enabled	HIGH
IAM-002	Access key older than 90 days or unused for 90 days	MEDIUM
IAM-003	IAM policy granting admin-level permissions (*:*)	HIGH
RDS-001	RDS instance publicly accessible	HIGH
RDS-002	RDS instance with encryption disabled	MEDIUM
EC2-001	EC2 instance with public IP and no WAF or load balancer	MEDIUM
LAMBDA-001	Lambda function with overly permissive IAM execution role	MEDIUM

6.2 Attack-Path Correlation

This is AetherOps's primary differentiator. Most scanners report findings as isolated items. AetherOps checks whether related findings form a realistic path an attacker could follow through the environment.

How it works:

- Build a directed graph using NetworkX from the relationships table. Nodes are resources; edges represent access relationships.
- Start BFS traversal from nodes with HIGH-severity findings.
- Follow edges to connected resources that also have findings.
- If a traversal produces a chain of two or more findings, record it as an attack path.
- Store the path as an ordered JSON list of resource IDs in the attack_paths table.
- The AI report narrates each path in plain English.

Example path: Stale IAM key (IAM-002) → EC2 instance with open SSH (SG-001) → public S3 bucket (S3-001). AetherOps narrates this as: 'An attacker with the unused key AKIA... can reach server i-0abc via SSH, and from that server can access the public bucket my-data. Rotate the key immediately and restrict port 22 to eliminate this path.'

6.3 Role-Based Access Control

Role	Permissions	Typical User
Admin	Full access: trigger scans, manage users, resolve/ignore findings, view all data	Project owner / lead developer
Auditor	Read all findings + audit log. Can resolve or ignore findings. Cannot manage users	Security reviewer / team member
Viewer	Read-only dashboard access. Cannot see audit log or trigger actions.	Stakeholder / client demo

6.4 Audit Trail

Every important action in the system writes an append-only record to the audit_log table: user ID, action type, target resource or finding, and timestamp. The log cannot be edited or deleted through any API endpoint. Admins and Auditors can view the full log on the dashboard.

7. AI Domain

7.1 AI-Generated Full Report

When the user clicks Generate Report, the backend assembles a structured context payload from the database and sends it to Gemini in a single prompt. Gemini returns a structured JSON report that the dashboard renders into sections.

What goes into the prompt:

- All security findings with severity, rule, and resource name
- All detected attack paths as ordered lists of resource descriptions
- Billing summary: monthly cost per service, top 3 cost drivers
- Resource inventory summary: counts and types
- Live pricing comparison data: AWS vs Azure vs GCP for the user's instance types
- Instruction: return a structured JSON with specific sections (see below)

Report output structure:

Report Section	Content
`security_summary`	Overall posture score (High / Medium / Low risk), total findings count, top three issues
`attack_paths`	Array of narrated paths, each with a plain-English description and recommended first action
`cost_analysis`	Monthly spend, day-over-day trend commentary, idle resource flags
`cloud_comparison`	Side-by-side cost estimate for current workload on AWS, Azure, GCP with AI commentary
`recommended_actions`	Prioritised list of actions sorted by impact, each with an estimated fix time

7.2 Contextual Chat Bot

After the report is generated, a chat widget appears on the dashboard. The user can ask follow-up questions in natural language. What makes this genuinely useful — rather than a generic AI chat — is that every message includes the full report JSON in Gemini's system prompt. The model can only answer from that data.

Example interactions:

User	What is my most dangerous resource right now?
Aether Ops AI	Your EC2 instance i-0abc123 is the highest-risk resource. It has an open SSH port reachable from the internet and is accessible via a stale IAM key that has admin permissions. Fixing either of these breaks the attack path.
User	Would migrating to Azure save money?

Aether Ops AI	Based on your current usage, your t3.medium instances would map to Azure B2s. At current pricing, the cost difference is approximately 8% in Azure's favour for your eu-west-1 equivalent region. However, your S3 usage pattern would cost slightly more on Azure Blob — net effect is roughly neutral. GCP is slightly cheaper overall for your workload profile.
User	How do I fix the public S3 bucket?
Aether Ops AI	Go to S3 → your-bucket-name → Permissions → Block Public Access → enable all four options. Then review the bucket policy and remove any statements that grant s3:GetObject to Principal: *.

Implementation note: the chat bot does not maintain conversation history across sessions — it is scoped to the current report and the current browser session. This keeps implementation simple and the context window manageable. Each message sends: [system: full report JSON] + [conversation history so far] + [new user message].

8. Technology Stack

Component	Technology	Why This Choice
Frontend	React + Tailwind CSS	Component-based, fast to build dashboards with. Tailwind avoids writing custom CSS fo
Backend	FastAPI (Python)	Lightweight, async-capable, and generates automatic API docs. Strong validation via Py
Database	PostgreSQL	Relational — handles both structured resource data and graph-style relationships clean
ORM	SQLAlchemy	Keeps database queries in Python, reduces raw SQL errors, easy migrations.
AWS SDK	boto3	Official Python SDK. Well-documented, covers every service we need.
Graph Analysis	NetworkX	Python library for graph traversal. BFS/DFS on our relationships table is straightforward
Authentication	PyJWT + bcrypt	JWT for stateless auth, bcrypt for password hashing. Both are industry-standard.
AI Model	Google Gemini API	Free tier is generous enough for report generation. Handles large context well.
Pricing APIs	AWS / Azure / GCP Pricing APIs	All are free with no key. GCP requires a free API key. All return JSON.
Containerisation	Docker	Ensures consistent environment across all four team members' machines.
CI/CD	GitHub Actions	Automated testing and deployment on every push. Free for student projects.
Cloud (Testing)	AWS (sandbox account)	Real AWS account with deliberately planted misconfigurations for integration testing.

9. Database Schema

Nine tables. Each serves a single purpose with no duplication.

Table	Key Fields	Purpose
`resources`	`id, arn, type, region, account_id, name, attributes (JSON)`	Every discovered AWS resource. attributes stores type-specific fields as JSON.
`relationships`	`id, resource_a_id (FK), resource_b_id (FK), relation_type`	Connections between resources. Drives the attack-path graph.
`findings`	`id, resource_id (FK), rule_id, severity, status, note, timestamp`	One row per rule violation. status: open / resolved / ignored.
`attack_paths`	`id, finding_chain (JSON), severity, created_at`	finding_chain is an ordered list of resource IDs forming the path.
`costs`	`id, service_name, month, amount, currency`	Monthly cost per AWS service from Cost Explorer.
`cloud_pricing`	`id, provider, instance_type, region, hourly_price, fetched_at`	Cached pricing from AWS, Azure, GCP APIs. Refreshed on each report generation.
`rules`	`id, rule_id, title, severity, resource_type, condition (JSON)`	Security rules loaded from YAML. Stored in DB for dashboard display.
`users`	`id, email, password_hash, role, created_at`	Dashboard user accounts. role: admin / auditor / viewer.
`audit_logs`	`id, user_id (FK), action, target_id, detail, timestamp`	Append-only. No update or delete endpoint exists for this table.

10. API Reference

All endpoints require JWT authentication except POST /auth/login. Role restrictions are noted in the Access column.

Method	Path	Access	Purpose
POST	`/auth/login`	Public	Authenticate user; returns JWT token
POST	`/auth/register`	Admin	Create a new user account with a role
GET	`/resources`	Admin/Auditor	List all discovered resources
POST	`/scan/trigger`	Admin	Start a new AWS discovery + evaluation scan
GET	`/findings`	All roles	List findings, filterable by severity and status
POST	`/findings/{id}/resolve`	Admin/Auditor	Mark a finding resolved with a reason note
POST	`/findings/{id}/ignore`	Admin/Auditor	Ignore a finding with a reason note
GET	`/attack-paths`	All roles	Return all detected attack paths
GET	`/attack-paths/{id}`	All roles	Get a single attack path with resource details
GET	`/costs`	All roles	Monthly cost breakdown by service
GET	`/pricing/compare`	All roles	Fetch live cross-cloud pricing comparison
POST	`/report/generate`	Admin/Auditor	Trigger AI report generation; returns report JSON
GET	`/report/latest`	All roles	Retrieve the most recent generated report
POST	`/chat/message`	All roles	Send a chat message; receives AI response grounded in report
GET	`/audit-logs`	Admin/Auditor	View the full audit trail
POST	`/rules/upload`	Admin	Upload a new YAML rules file

11. Implementation Plan

The project is structured as a six-week sprint. Each phase has a clear deliverable that can be demonstrated independently, so progress is visible and problems surface early.

Phase	Focus	Deliverables	Owners
Week 1	**Foundation**	DB schema, Docker setup, GitHub Actions CI, user auth with JWT and bcrypt, basic FastAPI structure.	Dhruv, Tushya
Week 2	**Resource Discovery**	boto3 integration, assume-role via STS, EC2/S3/RDS/IAM/Lambda discovery, relationships table population, ingestion API.	Janki, Priyanshi
Week 3	**Security + Billing**	Rules Engine with YAML loader, 10 rules implemented, findings API, Cost Explorer integration, costs table.	All four
Week 4	**Attack Paths + Pricing**	NetworkX graph from relationships, BFS attack-path detection, attack_paths table, pricing API calls, SKU mapping table.	Dhruv, Janki
Week 5	**AI + Dashboard**	Gemini report generation prompt, report JSON schema, React dashboard: findings list, attack path view, cost chart, report section, chat widget.	Tushya, Priyanshi
Week 6	**Testing + Deployment**	Integration testing against sandbox AWS account with seeded misconfigurations, end-to-end demo, Docker deployment, final documentation.	All four

12. Feasibility Assessment

Every component is assessed honestly below. Effort is rated for a four-person student team working part-time alongside coursework. Risk reflects the main failure modes.

Component	Effort	Risk	Notes
Resource Discovery	Medium	Low	AWS APIs are well-documented. boto3 handles pagination. Main effort is mapping relationships correctly.
Rules Engine	Low	Low	Straightforward condition checks against resource attributes. YAML format keeps rules easy to add.
Attack-Path Graph	Medium-High	Medium	The hardest component. NetworkX handles BFS easily but the traversal logic needs careful design to avoid false paths. Requires complete relationships data from discovery.
Billing Integration	Low	Low	Cost Explorer API is well-documented. Single API call returns all needed data.
Multi-Cloud Pricing	Medium	Low	AWS and Azure APIs need no key. GCP needs a free API key. Main effort is building and validating the SKU mapping table.
AI Report Generation	Medium	Medium	Prompt engineering to get structured JSON back reliably. Risk: Gemini returns inconsistent format. Mitigation: strict JSON schema in prompt, retry logic.
Chat Bot	Low-Medium	Low	Injecting report context into system prompt is simple. Main risk is context window size if report is very large — mitigated by summarising long sections.
RBAC + Auth	Low	Low	PyJWT and bcrypt are standard. Role checks on every endpoint are mechanical.
Frontend Dashboard	Medium	Medium	React with Tailwind is fast to build. Risk is time — six sections of UI take longer than expected. Mitigation: build with mock data first, connect to API last.
Docker + CI/CD	Low	Low	GitHub Actions free tier covers student projects. Docker ensures environment consistency.

Overall verdict: all components are feasible within a six-week student sprint. The attack-path graph is the most technically complex piece — it should be started in Week 4 only after the relationships table is fully populated and tested. The multi-cloud pricing comparison is simpler than it appears because the data is freely available via API — the main work is the one-time SKU mapping table.

13. What This Demonstrates

AetherOps is deliberately designed to showcase skills across three domains that are increasingly valued in the industry. The table below maps each feature to the specific skill it demonstrates for a recruiter or hiring manager.

Cloud Domain	
AWS IAM + boto3	Cross-account access via read-only IAM role, STS AssumeRole, least-privilege design
Resource Discovery	Programmatic inventory of EC2, S3, RDS, IAM, Lambda at scale
Cost Engineering	AWS Cost Explorer integration, cost-per-service analysis, idle resource detection
Multi-Cloud Awareness	Live pricing API integration across AWS, Azure, GCP with SKU equivalency mapping
Cloud Security Concepts	CSPM principles, security group rules, IAM policy analysis, encryption compliance

Security Domain	
CSPM	Cloud Security Posture Management — the discipline AetherOps implements
Misconfiguration Detection	Rule-based evaluation engine, YAML-defined rules, severity classification
Attack-Path Analysis	Graph traversal to link findings into realistic attack chains
RBAC	Role-based access control with JWT, least-privilege enforcement throughout
Audit Logging	Append-only, tamper-resistant event trail — a compliance requirement in real systems

AI Domain	
LLM Integration	Gemini API for structured report generation and natural language narration
Prompt Engineering	Structured prompts that return reliable JSON from an LLM
Grounded AI	Chat bot answers scoped strictly to user data — not generic responses
AI Orchestration	Multi-step pipeline: data aggregation → pricing fetch → AI generation → structured output
Human-Readable Security	Translating technical findings into plain-language explanations — a real product skill

How this reads on a resume:

"Built AetherOps, a cloud security posture management platform for AWS that automatically discovers resources via read-only IAM role, evaluates them against a YAML-defined security rule set, correlates findings into attack paths using graph traversal (NetworkX), and generates a full AI-narrated security and cost report with cross-cloud comparison (AWS, Azure, GCP) using live pricing APIs. Includes a grounded AI chat bot scoped to the user's own account data. Stack: FastAPI, PostgreSQL, React, boto3, Gemini API. Deployed via Docker + GitHub Actions."

This is a project that covers Cloud + Security + AI in one coherent system, is demonstrably useful (not just a tutorial clone), and is scoped honestly for a student team to actually finish. That combination is rare and will stand out.